

FMCW MIMO Radar 3-D FFT Accelerator

Complete Project Report — Specification, Architecture, Build History,
Verification Status, and Open Items

Vlsi Integration

3 September 2026

Contents

1	1. Project Overview	3
1.1	1.1 What this project is	3
1.2	1.2 Target hardware	3
1.3	1.3 Collaboration split	3
2	2. Frozen Specification	5
2.1	2.1 Data cube	5
2.2	2.2 Arithmetic and numerics	5
2.3	2.3 Interface / bus architecture	5
2.4	2.4 Block-floating-point exponent bookkeeping	6
2.5	2.5 Related earlier-stage specification (Stage-1 Range engine)	6
3	3. Architecture — How It Was Built	8
3.1	3.1 Top-level pipeline (per the current RTL, radar_dsp_3d_top.sv)	8
3.2	3.2 Design decisions and their reasoning	8
3.3	3.3 Corner-turn interface contract (for the collaborator’s real block)	9
4	4. The Memory / Bandwidth Problem (Why the Corner Turn Is Hard)	11
4.1	4.1 The cube does not fit on chip	11
4.2	4.2 Board selection was driven by this, not by preference	11
4.3	4.3 Bandwidth requirement and measurement	11
4.4	4.4 Clock-domain crossing — a requirement discovered late, now documented before it causes damage	11
5	5. Build Chronology (20-Day Log, Condensed)	13
6	6. What Has Been Built and Verified (With Numbers)	15
6.1	6.1 Functional simulation — PASS	15
6.2	6.2 Numerical correctness — proven, not just “it ran”	15
6.3	6.3 Protocol correctness	15
6.4	6.4 Elaboration — clean at full spec	16
6.5	6.5 Synthesis — clean, with real timing closure	16
7	7. Problems Faced and How Each Was Solved	17
7.1	7.1 The pattern worth naming	21
8	8. What Is Currently Working	23

9	9. What Is Pending — Stated Plainly	24
9.1	9.1 The corner turn itself — the largest remaining item	24
9.2	9.2 Verification gaps	24
9.3	9.3 System-level items not yet started at all	24
9.4	9.4 Noted but deliberately not yet acted on	25
10	10. Corner Cases and Known Limitations (Consolidated)	26
11	11. Repository Structure (Current State)	28
12	12. Summary Assessment	29
13	13. Technical Appendix — Formulas, Derivations, and Block-Level Detail	30
13.1	13.1 Data cube size and bandwidth — full derivation	30
13.2	13.2 Q1.15 fixed-point format and windowing arithmetic	31
13.3	13.3 Range/Doppler FFT core configuration (Xilinx xfft v9.1)	32
13.4	13.4 Angle FFT — full closed-form 4-point DFT (rtl/angle_fft4_par.sv)	33
13.5	13.5 Magnitude-squared arithmetic (rtl/complex_mag2.sv)	34
13.6	13.6 CA-CFAR detector arithmetic (rtl/ca_cfar.sv)	34
13.7	13.7 Block-floating-point exponent accumulation, exact bit path	35
13.8	13.8 4-lane packing address formula and DDR layout (rtl/axis_pack4.sv, rtl/ctm_stub.sv)	35
13.9	13.9 Synthesis resource-count cross-check	36
13.10	13.10 Timing budget arithmetic	36
13.11	13.11 Clock-domain-crossing frequency relationship (Section 4.4, restated arithmetically)	37

1. Project Overview

1.1 What this project is

This is a hardware accelerator for an FMCW (Frequency-Modulated Continuous-Wave) MIMO radar signal-processing chain, implemented as synthesizable RTL (SystemVerilog) targeting a Xilinx Kintex-7 FPGA. The design performs the standard 3-D radar cube processing pipeline in hardware:

Range FFT -> Corner Turn (transpose) -> Doppler FFT -> Angle FFT -> $|X|^2$ -> CA-CFAR

Each stage corresponds to a physical measurement the radar extracts from raw ADC samples:

- **Range FFT** — resolves distance to a target from the beat frequency within one chirp.
- **Corner turn** — transposes the data cube from range-major to chirp-major order so the Doppler FFT can operate coherently across chirps for a fixed range bin. This is a memory-layout operation, not a numerical one, but it is the single hardest engineering problem in the whole project (see Section 4).
- **Doppler FFT** — resolves target radial velocity from phase change across consecutive chirps at a fixed range bin.
- **Angle FFT** — resolves target angle-of-arrival from phase difference across the 4 receive (RX) antennas (a 4-element virtual/physical array).
- **Magnitude-squared ($|X|^2$)** — converts complex bins to real power for detection.
- **CA-CFAR** (Cell-Averaging Constant False Alarm Rate) — adaptive thresholding that flags detections against local noise floor rather than a fixed threshold.

1.2 Target hardware

Item	Value
Board	Xilinx Kintex-7 KC705
Device	XC7K325T-2FFG900C
Logic budget	203,800 LUT / 407,600 FF / 840 DSP48E1 / 445 RAMB36 (890 RAMB18)
On-board DRAM	DDR3, via Xilinx MIG (Memory Interface Generator)
Target clock	100 MHz (ac1k) for the DSP pipeline

The board was **not** the original target. The project began on Zynq-7000 (Zybo Z7-20 / PYNQ-Z2) devices and moved to KC705 specifically because the data-cube size does not fit in on-chip memory on any candidate device — the KC705 was chosen for its larger BRAM, DSP count, and a 64-bit DDR3 path via MIG (see Section 4.2 and Day 5 of the build log, Section 6).

1.3 Collaboration split

Two engineers are building this system:

- **This side (verified in this report):** Range FFT, Doppler FFT, Angle FFT, magnitude, CA-CFAR, the 4-lane packing/unpacking infrastructure, block-floating-point exponent tracking, the full elaboration/synthesis build flow, and the interface contract for the corner turn.
- **Collaborator (“bhaiya”):** the real DDR-backed corner-turn (CTM) block, built against a written contract (rtl/ctm_stub.sv header + tb/tb_ctm_transpose.sv acceptance test)

rather than free-form integration. This report documents exactly what that contract requires, including a clock-domain-crossing requirement discovered during this reporting cycle that had not previously been written down anywhere (Section 4.4).

2. Frozen Specification

All values below were fixed on Day 1 of the build and are treated as a hard contract — not a starting point for negotiation. RTL module defaults are the spec; any smaller test size must be requested explicitly via parameters, and an elaboration-time assertion ($N_RANGE == 2^{RANGE_NFFT}$, $N_CHIRP == 2^{DOPPLER_NFFT}$) enforces that the two never silently disagree.

2.1 Data cube

Parameter	Frozen value
Range bins	1024
Doppler bins (chirps per CPI)	256
RX antennas	4
Sample format	Q1.15 complex, 16-bit I / 16-bit Q
Cube size	1024 x 256 x 4 x 4 bytes = 4.19 MB per cube
Ping-pong cube (process frame k while k+1 arrives)	8.39 MB

This exact figure (1024x256x4) was the subject of a spec-drift bug found on Day 20 (Bug #12, Section 7) — the build had silently regressed to 16 chirps / 128-point Doppler. It is called out here specifically because it was the trigger for the cleanup and hardening work described in this report.

2.2 Arithmetic and numerics

Parameter	Frozen value
Fixed-point format	Q1.15 (1 sign + 15 fraction bits), 16-bit I and Q
Scaling strategy	Block Floating Point (BFP)
Rounding	Convergent (round-half-to-even)
FFT core	Xilinx LogiCORE xfft v9.1 , Pipelined Streaming I/O architecture
Radix	Radix-2
Output ordering	Natural order (not bit-reversed)
Twiddle factors	16-bit Q1.15

BFP was chosen deliberately over fixed scaling: a radar scene has enormous dynamic range between a nearby wall and a distant target, and a single fixed scale factor either clips the near return or buries the far one in quantization noise. BFP tracks one shared exponent per block and rescales the mantissa, at the cost of having to carry that exponent (BLK_EXP) correctly through every downstream stage — this bookkeeping is one of the most error-prone parts of the design (Section 2.4).

2.3 Interface / bus architecture

Parameter	Frozen value
Antenna handling	4 parallel lanes , not time-multiplexed
Internal bus	128-bit AXI4-Stream , all 4 antennas of one range/chirp cell packed into a single beat

Parameter	Frozen value
Streaming protocol	AXI4-Stream discipline: TVALID, TREADY, TLAST, full backpressure at every block boundary
Clock	100 MHz single clock domain for the DSP pipeline (aclk/aresetn)
Throughput	4 lanes x 1 sample/clock = 400 MSPS aggregate at 100 MHz (required: ~102.4 MSPS)

The 4-lane, single-bus-beat packing is a load-bearing architectural decision, not a convenience: if the four antennas travelled as four independent streams, they could drift by a beat relative to each other and the angle transform would silently combine antenna samples from *different* range/chirp cells. Packing them into one TVALID/TREADY handshake makes that class of bug structurally impossible rather than something that has to be verified by test. See Day 16, Section 6.

2.4 2.4 Block-floating-point exponent bookkeeping

The BFP exponent accumulates additively as data passes through each transform stage, and must be tracked explicitly because a single register is not sufficient once frames overlap in a pipelined datapath:

```

after Range FFT      value x 2Er
after Doppler FFT    value x 2(Er+Ed)      (Doppler core reports only Ed)
after Angle FFT      value x 2(Er+Ed+2)    (divide-by-4 inside angle_fft4_par)
after |X|^2          value x 2(2*(Er+Ed+2)) (squaring doubles the exponent)

```

exp_accum.sv exists specifically to carry the Range stage's BLK_EXP across the Doppler stage's pipeline latency, because a naive single-register carry breaks under overlapping frames.

2.5 2.5 Related earlier-stage specification (Stage-1 Range engine)

An earlier, narrower specification document (SPEC_FROZEN.md) froze the Stage-1 Range-FFT engine in isolation, before the full 3-D chain was designed, and is retained as the integration contract for that stage. Values of note that still apply:

Parameter	Value	Note
FFT core	Xilinx FFT IP v9.1, Pipelined Streaming I/O, Radix-2	matches Section 2.2
Header layout	BLK_EXP[7:0], LANE_ID[15:8], NFFT[20:16], FRAME_COUNT[63:32]	bit-exact, frame header format
Frame alignment	32-byte aligned, frame bytes = 32*(1+N/8)	N=1024 -> 4128 bytes; N=2048 -> 8224 bytes
DDR write rate at N=1024	403.125 MB/s, 50.4% of a 64-bit bus	derived, not chosen

That document also flagged an **unresolved architectural conflict** worth restating here because it is still open: Stage-1's frozen output format is 32-bit unsigned magnitude-squared,

but Doppler and Angle processing are coherent transforms that require **phase**, which $|X|^2$ discards irrecoverably. The proposed resolution — a mode bit in a reserved header field selecting between magnitude-squared output (mode 0, default) and complex $\{\text{Im}, \text{Re}\}$ output (mode 1, for Stage 2 consumption) — was recorded as a decision pending sign-off and has since been superseded in practice by the 3-D top-level design, which keeps data complex internally between the Range and Doppler stages and only takes the magnitude after the Angle FFT. This is noted here for completeness and traceability, not because it blocks anything in the current design.

3. Architecture — How It Was Built

3.1 Top-level pipeline (per the current RTL, radar_dsp_3d_top.sv)

ADC-format input (4 antennas, packed 128-bit beat)

|

v

[window_lane x4] Hanning window, fixed 1024-pt ROM, DSP48-inferred multiply

|

v

[fft_lane x4] Range FFT (xfft v9.1, runtime NFFT_SEL -> 1024-pt)

|

v

[ctm_stub / real CTM] Corner turn: range-major -> chirp-major transpose (DDR-backed)

|

v

[doppler_lane x4] Doppler FFT (same xfft v9.1 core, runtime NFFT_SEL -> 256-pt)

|

v

[angle_fft4_par] 4-point Angle FFT across antennas, combinational, 0 DSP / 0 BRAM

|

v

[complex_mag2 x4] $|X|^2$, 3-stage pipelined multiply-accumulate

|

v

[ca_cfar x4] CA-CFAR adaptive threshold detection

|

v

[axis_skid x8] protocol-correctness registers (see Section 7, Bug #11)

|

v

[axis_pack4 / axis_unpack4] 128-bit bus (re-)packing at stage boundaries

|

v

Output detections

Confirmed instance counts at elaboration (full 1024x256x4 spec): 4x fft_lane, 4x doppler_lane, 1x ctm_stub, 1x angle_fft4_par, 4x complex_mag2, 4x ca_cfar, 3x axis_pack4, 3x axis_unpack4, 1x exp_accum, 4x window_lane, 8x axis_skid.

3.2 Design decisions and their reasoning

Why one shared xfft IP core for both Range and Doppler. The Xilinx xfft v9.1 core was generated once with run_time_configurable_transform_length=true up to N=2048, and is reused for both the 1024-point Range transform (NFFT_SEL -> 10) and the 256-point Doppler transform (NFFT_SEL -> 8). This avoids maintaining two separately-generated IP cores and allowed reduced-size simulation (16-point, Section 5) without regenerating anything. The trade-off is that every instance carries the BRAM cost of a 2048-point core regardless of which size it actually runs at — a known, documented, and currently-untaken optimization (Section 8).

Why 4 parallel lanes instead of 1 time-multiplexed lane. Evaluated explicitly on Day 16 against a single time-multiplexed lane:

	1 lane, time-multiplexed	4 lanes (built)
Throughput needed	102.4 MSPS, needs a clock >103 MHz	400 MSPS at 100 MHz
Angle FFT	needs buffering + reorder logic	combinational, 0 DSP, 0 BRAM
Second corner turn	required (to regroup antennas)	eliminated
DDR traffic	2052 MB/s	819 MB/s
DSP cost	~45	~180 of 840 (21%)

4 lanes was chosen: it removes an entire second corner-turn stage, keeps the clock at a comfortable 100 MHz rather than pushing past 103 MHz, and lets the Angle FFT become a zero-cost combinational operation because all 4 antenna samples for a cell are already present on the same bus beat.

Why Block Floating Point over fixed-point scaling. See Section 2.2 — driven by the dynamic range of real radar returns.

Why the corner turn must be DRAM-backed, not on-chip. See Section 4.2 — a hard resource-arithmetic conclusion, not a preference.

Why AXI4-Stream with a skid buffer at every boundary. Established on Day 2 as a project-wide rule: TVALID must never combinationaly depend on TREADY. This rule was violated once anyway, deep in the design (ca_cfar), and caused a full pipeline deadlock that took a 4-hypothesis investigation to root-cause (Section 7, Bug #11) — the rule existing on paper since Day 2 did not prevent the bug; only re-auditing every block against it did.

3.3 3.3 Corner-turn interface contract (for the collaborator’s real block)

Because the real corner-turn block is being built by a second engineer, its interface is specified as a written contract in `rtl/ctm_stub.sv`, independent of whichever placeholder currently occupies that slot in synthesis:

1. **Address function.** $\text{addr}(c, r) = c * N_RANGE + r$, shared and unambiguous between the write-side (range-major) and read-side (chirp-major) access patterns.
2. **Bus width rationale.** 128 bits, carrying all 4 antennas of one cell per beat — for two independent reasons: (a) it preserves antenna lockstep, the same reason the whole pipeline is 4-lane (Section 3.2); (b) DDR burst efficiency — measured data (Section 4.3) shows a 252x bandwidth swing between small and large burst sizes, so the corner turn must read/write in range-blocks, never single rows.
3. **Mandatory requirements:**
 - Full backpressure on both the write and read AXI-Stream ports.
 - Ping-pong double buffering (bank A / bank B) with independently tracked read and write bank registers, so the writer can fill frame $k+1$ while the reader drains frame k .
 - `BLK_EXP` passthrough via `tuser`, latched once per frame (constant across the frame, per the documented assumption in Section 2.4 and the caveat in Section 9).
 - `TLAST` correctly asserted at end-of-frame.
4. **Capacity constraint.** The 8.39 MB ping-pong cube cannot fit in the KC705’s 2.00 MB of on-chip BRAM in any arrangement (Section 4.2) — the real block must be DDR-backed via the already-generated MIG controller in `ctm_test/`, keeping only a 64–128 kB on-chip working slice, and must stream to/from DDR sequentially in range-blocks, not single rows.
5. **Clock-domain crossing (newly identified requirement — see Section 4.4).** The corner turn must cross between the 100 MHz `ac1k` DSP-pipeline domain and the 200

MHz `ui_clk` MIG domain internally; its AXI-Stream ports must remain on `aclk` from the rest of the pipeline's point of view.

The placeholder module (`ctm_stub.sv`) itself has two bodies selected by a ``ifdef SYNTHESIS` compiler directive: a synthesis-time body that is a 1-deep register (full backpressure/tuser/tlast passthrough, but **not** a transpose — it exists purely so the rest of the pipeline can elaborate and synthesize around this instance without exceeding Vivado's flattened-array elaboration ceiling at real cube sizes), and a simulation-time body that is a true dual-bank behavioral transpose model, exercised by `tb/tb_ctm_transpose.sv`.

4 4. The Memory / Bandwidth Problem (Why the Corner Turn Is Hard)

This section is presented on its own because, per the build log, it is the single design decision that everything downstream is a consequence of.

4.1 4.1 The cube does not fit on chip

1024 range x 256 Doppler x 4 antennas x 4 bytes = 4.19 MB per cube
Ping-pong (process frame k while frame k+1 arrives) = 8.39 MB

Available on-chip Block RAM: 0.6 MB on Zynq-7020, **2.0 MB on Kintex-7 325T (KC705)**. The corner turn between Range and Doppler cannot live entirely on-chip — not at these dimensions, not on either candidate device, in no tiling arrangement. This was proven by resource arithmetic (Day 4), not assumed.

4.2 4.2 Board selection was driven by this, not by preference

Zybo Z7-20, PYNQ-Z2, and KC705 were compared explicitly. KC705 was selected for 2.0 MB BRAM, 840 DSP48E1, and a 64-bit DDR3 interface via MIG, versus the Zynq boards' 32-bit path. The honest framing recorded at the time: **the bigger board does not make the cube fit on-chip — it makes the DRAM-backed path fast enough that not fitting stops mattering.**

4.3 4.3 Bandwidth requirement and measurement

Required bandwidth:

$$\begin{aligned} \text{BW} &= 2 \times \text{bytes_per_sample} \times N_{\text{rx}} \times f_{\text{sample}} \\ &= 2 \times 4\text{B} \times 4 \times 25.6 \text{ MSPS} \\ &= 819 \text{ MB/s} \end{aligned}$$

N_{range} and N_{chirp} cancel out of this formula exactly — **bandwidth requirement is independent of transform size**. A larger FFT means more data per frame, but proportionally lower frame rate; you cannot reduce required bandwidth by shrinking the transform.

Real DDR bandwidth was measured on PYNQ-Z2 hardware (the *weaker* of the candidate boards) at **1541 MB/s**, comfortably above the 819 MB/s requirement. Critically, access pattern was also measured and found to dominate: 128-byte transfers achieved 6 MB/s, while 16 MB transfers achieved 1539 MB/s on the same hardware — **a 252x swing from access pattern alone**. This single measurement is the reason the corner-turn contract (Section 3.3) mandates range-block reads rather than single-row reads: a naive row-at-a-time implementation could be off the required bandwidth by two orders of magnitude despite the DRAM itself having ample headroom.

4.4 4.4 Clock-domain crossing — a requirement discovered late, now documented before it causes damage

While answering a completeness question during this reporting cycle, the generated MIG project file (`ctm_test/ctm_bandwidth_test.gen/.../mig_ddr3_0/mig.prj`) was inspected directly rather than assumed, and revealed:

MIG parameter	Value	Implication
TimePeriod	1250 ps	800 MHz DDR3-1600 memory clock
PHYRatio	4:1	-> ui_clk = 200 MHz
DataWidth	64 bits	12.8 GB/s theoretical peak vs. 819 MB/s required — bandwidth headroom is not the risk

The DSP pipeline runs at 100 MHz (aclk), and post-synthesis Fmax for the whole pipeline is only about 182 MHz (Section 6, Day 20) — running the entire design at 200 MHz to match ui_clk is not viable. **This means the corner turn is not just a memory-capacity problem, it is also a clock-domain-crossing problem**, requiring asynchronous FIFOs (or an equivalent CDC-safe structure) inside the corner-turn block itself, with the AXI-Stream ports to the rest of the pipeline staying on aclk.

This had never been written down anywhere before this reporting cycle. It has now been added as a new, explicit section in the corner-turn contract (rtl/ctm_stub.sv header, Section 3.3 above) specifically so the collaborator is aware of it *before* starting the real implementation, rather than discovering it during integration. It is flagged plainly: this module’s current single-aclk-port list is a simulation-model simplification; the real block needs a ui_clk port, its own reset synchronization, and CDC-aware timing constraints (e.g. set_max_delay -datapath_only across the crossing). The existing tb_ctm_transpose.sv testbench is single-clock and tests only the transpose *addressing* contract — it does **not** exercise the CDC and cannot be relied on to catch CDC bugs. This is currently the largest unverified risk in the whole project (see Section 9).

5 5. Build Chronology (20-Day Log, Condensed)

Day	Milestone
1	Specification freeze: 1024x256x4, Q1.15, xfft v9.1, BFP, convergent rounding
2	Range-lane architecture; AXI-Stream skid-buffer rule established (TVALID must never depend on TREADY)
3	Cube-size problem identified: 4.19 MB per cube vs. 0.6–2.0 MB on-chip BRAM
4	Resource-arithmetic confirmation: corner turn must be DRAM-backed
5	Board evaluation; KC705 selected
6	Bandwidth requirement derived (819 MB/s, size-independent); real hardware measurement (1541 MB/s, 252x access-pattern swing)
7	Design-state consolidation and documentation
8	Build infrastructure: Tcl scripts for IP generation, synthesis, reporting
9	Vivado license blocker diagnosed (device not covered by free tier, not a network fault); resolved via institutional license server
10	Migrated FFT IP + generated MIG DDR3 controller for KC705
11	First real RTL: Hanning window, fft_lane, axis_skid, config FSM, status drain
12	First (later superseded) 4-lane experiment and tiled corner-turn attempt
13	NumPy golden-model and simulation infrastructure built (dimensions later found to have drifted from spec — see Bug #14, fixed 3 Sep)
14	Full code review — 4 critical bugs found (angle “FFT” was a shift register; corner turn had no working transpose in either of 2 attempts; the “passing” testbench checked beat count, not data; synthesis targeted the wrong silicon device)
15	Bug fixes: Doppler config FSM, complex_mag2 DSP duplication, ca_cfar threshold shift hardcoding, new exp_accum module
16	4-lane architecture decision (Section 3.2), axis_pack4/angle_fft4_par built
17	Full-chain integration in radar_dsp_3d_top.sv; corner-turn contract written for collaborator; BLK_EXP bookkeeping documented

Day	Milestone
18	Synthesis blocked by unreachable license server (network issue, diagnosed via Test-NetConnection); first true end-to-end testbench written against real xfft IP
19	The deadlock hunt — 4-hypothesis investigation, root cause found by instrumentation, not guessing (Section 7, Bug #11); result: PASS, 263 checks, 0 errors
20	Spec-drift bug found by asking a direct question (Bug #12); repository reorganized and cleaned; first real synthesis numbers obtained ; blank-WNS false-success bug found and fixed (Bug #13)

6 6. What Has Been Built and Verified (With Numbers)

6.1 6.1 Functional simulation — PASS

The end-to-end testbench (tb/tb_radar_dsp_3d_top_smoke.sv, run via scripts/run_pipeline_smoke.tcl) exercises the real Xilinx xfft IP core (not a behavioral stand-in) through the full chain.

Result: 263 checks, 0 errors. All ten internal pipeline boundaries reported 256/256 beats matching:

1. top input	: 256	2. Range FFT out	: 256
3. corner turn in	: 256	4. corner turn out	: 256
5. Doppler FFT out	: 256	6. Angle FFT out	: 256
7. $ X ^2$ out	: 256	8. CFAR out	: 256
9. skid out	: 256	10. top output	: 256

Note on test scale: this run was executed at a reduced 16-range x 16-Doppler size, not the full 1024x256 spec, because the corner-turn *placeholder's* simulation model (a flat behavioral array) cannot elaborate at the full cube size without exceeding Vivado's roughly 1,000,000-bit array elaboration ceiling — confirmed directly on Day 17/18. This is a limitation of the placeholder, not of the surrounding pipeline, and is discussed as an open verification gap in Section 9.

6.2 6.2 Numerical correctness — proven, not just “it ran”

Rather than trust a pass/fail count alone, two independent numerical properties were checked:

Real-input conjugate symmetry. For a real-valued input, the DFT must satisfy $X[k] = X^*[N-k]$, hence $|X[k]|^2 = |X[N-k]|^2$. The measured range-bin power profile:

range 0 : 2119936	
range 1 : 652544	range 15 : 652544
range 2 : 153856	range 14 : 153856
range 3 : 62720	range 13 : 62720
range 4 : 28928	range 12 : 28928
range 5 : 40192	range 11 : 40192
range 6 : 24832	range 10 : 24832
range 7 : 21760	range 9 : 21760
range 8 : 20736	(Nyquist bin, self-paired)

Every one of the seven mirror pairs matches **exactly**. This is a numerical correctness check on the Range FFT core's actual behavior, not merely a structural “did data flow through” check.

Angle-transform exactness. A constant signal across all 4 antennas produces energy only in angle bin 0, with bins 1–3 reading exactly zero — the mathematically correct result for a 4-point DFT of a constant input, and confirmed as such (two of the seven initial “failures” during debugging were traced to wrong expectations in the testbench, not defects in the design — angle bins 1–3 reading zero, and range-bin leakage from the fixed 1024-point Hanning window being applied to a 16-point transform, are both correct/expected behavior, not bugs).

Doppler + corner turn correctness. Identical chirps at the (reduced-size) test input produce all Doppler energy concentrated at Doppler bin 0, as required.

6.3 6.3 Protocol correctness

No AXI-Stream deadlock, no protocol violation, no X-propagation observed in the passing run — notable specifically because a real deadlock (Bug #11, Section 7) was found and fixed ear-

lier in the same testing effort, so “no deadlock” here is a verified outcome, not an unexamined default.

6.4 6.4 Elaboration — clean at full spec

scripts/check_elab.tcl, run at the **full** 1024x256x4 spec (not reduced), elaborates cleanly against the real KC705 part with correct parameter binding (NFFT_SEL0=10, NFFT_SEL1=11 for Range; NFFT_SEL0=7, NFFT_SEL1=8 for Doppler) and the correct instance counts listed in Section 3.1.

6.5 6.5 Synthesis — clean, with real timing closure

Run at the full frozen spec (build_synth.tcl), out-of-context, part xc7k325tffg900-2 (confirmed correct — this corrects an earlier build-script error that had targeted an Artix-7 part instead, Bug #4):

Resource	Used	Budget	Utilization
LUT	34,200	203,800	17%
FF	57,132	407,600	14%
DSP48E1	216	840	26%
RAMB18	146	890	16%
WNS @ 100 MHz	+4.505 ns	—	timing met

Critical path: 5.495 ns -> Fmax ~182 MHz post-synthesis (consistent with 162 MHz separately measured on this part during Stage-1 work). DSP breakdown checks out against a hand-estimate: 8 xfft cores contribute ~192 DSP48E1, plus 24 in fabric logic (8 window multipliers + 8 magnitude-squared + 8 CFAR threshold blocks) = 216, matching exactly.

What these numbers do and do not mean — stated explicitly so they are not over-read:

- **Do:** the design fits comfortably (a quarter of the device at most on any single axis) and closes the 100 MHz timing target with 45% margin.
- **Do NOT** treat 182 MHz as an achievable operating frequency — this is post-synthesis only, not post-place-and-route; routing delay is not yet included and will consume part of that margin.
- **Do NOT** treat this as final device utilization — u_ctm in this build is the register placeholder (1 deep register, no transpose logic). The real DDR-backed corner turn plus its MIG controller will add substantial logic and its own demanding timing constraints on top of these numbers.
- HD.CLK_SRC is unset in out-of-context mode, so clock skew is not fully modeled in this result.

7 7. Problems Faced and How Each Was Solved

Thirteen bugs were found and fixed over the course of the project. They are listed in full because several of them are instructive about *how* they were found, which mattered as much as the fix itself.

#	Bug	How it was found	Resolution
1	angle_fft_lane.sv was a 4-deep shift register — no transform was actually being performed, and it would have produced plausible-looking output forever	Code review	Replaced with angle_fft4_par.sv, a real combinational 4-point DFT
2	Corner turn had no working transpose in either of two implementation attempts — one incremented read/write pointers in lockstep (no transpose at all); the other used mismatched address formulas that only agreed at 2 of 65,536 addresses	Code review	Superseded by the documented contract in ctm_stub.sv with a single shared addr(c, r) formula, and tb_ctm_transpose.sv as an explicit acceptance test
3	The “passing” testbench only checked total beat count ("3D PIPELINE PASSED! 65536/65536"), which would have passed against either broken transpose in Bug #2	Code review	Rewritten to check actual data values, plus the conjugate-symmetry and constant-input checks described in Section 6.2

#	Bug	How it was found	Resolution
4	build_synth.tcl targeted xc7a200tfbg676-2 (Artix-7) under a comment that claimed Kintex-7 — every utilization/timing number produced before this fix described a device not actually on the desk	Code review	Corrected to xc7k325tffg900-2 (verified against the KC705 datasheet)
5	doppler_lane instantiated fft_config_fsm unmodified from the Range lane, so it requested a 1024-point Doppler transform instead of 128 — the core would have stalled waiting for input that never arrives	Code review	Parameterized NFFT_SEL0/NFFT_SEL1 per instantiation
6	complex_mag2 computed both I^2 and Q^2 twice, inferring up to 4 DSP48 slices for what is logically 2 multiplies	Code review	Restructured into a clean 3-stage pipeline, halving DSP usage for this block
7	ca_cfar's noise-average divide was a hardcoded shift-by-3 regardless of the REF_CELLS parameter — changing reference-cell count would silently scale the detection threshold wrong, producing detections at the wrong rate without any obvious failure	Code review	Derived shift amount from $\$clog2(REF_CELLS)$ with a compile-time power-of-two check

#	Bug	How it was found	Resolution
8	ctm_stub's behavioral (simulation) array model exceeded Vivado's roughly 1,000,000-bit elaboration ceiling at real cube dimensions	Elaboration failure	Split the module via <code>`ifdef SYNTHESIS</code> into a synthesizable 1-deep register body and a separate simulation-only array-based body
9	Hanning window coefficient file (hanning_1024.mem) could not be found by <code>\$readmemh</code> — the path was relative to the simulator's run directory, not the project root, so all four Range lanes silently ran on unknown (X) values	Simulation (zero-output investigation)	Fixed the path; required two attempts because Vivado flattens exported data files to the run-directory root and drops the path prefix
10	fft_config_fsm's reset default hardcoded NFFT=10 regardless of its actual parameter, issuing one incorrect configuration write before self-correcting	Code review, during the same zero-output investigation as Bug 9	Reset default now derives from the module parameter

#	Bug	How it was found	Resolution
11	ca_cfar / axis_pack4 combinational deadlock: ca_cfar's pipeline only advanced else if (m_axis_tready) (its TVALID effectively gated by downstream TREADY), while axis_pack4's TREADY was {4{all_valid & m_axis_tready}} (gated by upstream TVALID) — a circular combinational dependency with no escape path from reset, producing exactly zero output beats at every transform size tested	Found by \$monitor instrumentation on the config handshake after two other, real-but-unrelated bugs (window-file path, config-FSM reset) were ruled out by direct hypothesis testing at 8-pt and 16-pt transforms	Inserted an axis_skid register between each ca_cfar output and the axis_pack4 output join — a skid buffer's ready is high when empty regardless of downstream state, and its valid is a true register, breaking the combinational loop
12	Build scripts (radar_dsp_3d_top.sv defaults, build_synth.tcl, check_elab.tcl) were configured for N_CHIRP=16/128-point Doppler — a leftover from when the corner-turn stub had to stay small enough to elaborate as a flat array — while the frozen spec calls for 256 chirps/256-point Doppler; the constraint that motivated the small size no longer existed but nobody had gone back and updated the dimensions	Asking a direct verification question out loud ("is our 3D 1024x256x4?") rather than trusting prior configuration	Corrected all three files to the frozen spec; module defaults are now the spec itself, and an elaboration-time assertion enforces $N_RANGE == 2^{RANGE_NFFT} / N_CHIRP == 2^{DOPPLER_NFFT}$ so this class of drift cannot recur silently

#	Bug	How it was found	Resolution
13	Timing constraints never applied — synthesis reported a blank WNS which reads as success but actually means “timing was never analyzed.” Root cause was rtl/kc705_timing.xdc failing three independent ways at once: (a) a UTF-8 byte-order-mark on line 1 made Vivado reject the entire file (Command '<BOM>#' is not supported); (b) it constrained ports named sys_clk_p/sys_rst_n that do not exist on this module (its actual ports are aclk/aresetn), so create_clock silently matched nothing; (c) it requested 250 MHz against an architecture explicitly costed and sized for 100 MHz	Reading the actual printed output line-by-line instead of trusting the “0 errors” exit status	Wrote a new out-of-context-specific constraint file (constraints/ooc_radar_dsp_3d_top with correct port names, no BOM, and the correct 100 MHz clock; hardened build_synth.tcl so it can never again print a blank WNS silently — it now explicitly prints *** NO CLOCK DEFINED -- TIMING NOT ANALYSED *** if no clock exists, or *** NEGATIVE SLACK -- TIMING NOT MET *** if slack is negative

14 | scripts/golden_model_3d_fft.py — the project’s only floating-point reference model — was still parameterized N_range=512, N_doppler=128, N_angle=4, a stale value left over from before the Day-1 spec freeze to 1024x256x4. It ran without error and produced a plausible-looking golden_cube.npy, so nothing about running it would have revealed that it no longer matched the actual design | Found during preparation of this report, by checking the golden-model script’s parameters against the frozen spec rather than assuming a script that runs cleanly is still correct | Corrected to N_range=1024, N_doppler=256, N_angle=4; re-run and confirmed it now produces a 1024x256x4 complex128 cube (16.78 MB) end-to-end through all three FFT stages without error |

7.1 7.1 The pattern worth naming

Several of the most serious bugs (1, 2, 3, 11) share a root cause: a test or a build step *looked* successful (a passing beat-count test, a clean synthesis exit code) while the thing that actually mattered (real data correctness, real timing analysis) had not been checked at all.

The corrective action taken each time was the same: strengthen the check itself (data-value comparison instead of beat count; explicit WNS/clock-existence reporting instead of trusting exit code) so the same category of false success cannot recur, rather than only fixing the one instance found.

8. What Is Currently Working

- **Full 4-lane pipeline, functionally correct end-to-end** against the real Xilinx xfft IP core (not a simulation stand-in), verified by 263/263 passing checks with zero errors across all ten internal stage boundaries.
- **Numerically correct Range FFT**, proven via exact real-input conjugate symmetry ($|X[k]|^2 = |X[N-k]|^2$) across seven independent mirror-pair checks.
- **Numerically correct Angle transform**, proven via the expected exact-zero result at angle bins 1-3 for a constant-amplitude input across antennas.
- **Numerically correct Doppler stage and corner-turn addressing** (at reduced test scale), proven via all-energy-at-bin-0 for identical chirps.
- **No AXI-Stream protocol violations, no deadlock, no X-propagation** in the current design — actively verified, not merely assumed, since a real deadlock was found and fixed in the same effort (Bug 11).
- **Clean elaboration at the full frozen spec** (1024x256x4) against the real KC705 part, with all parameter bindings and instance counts verified correct.
- **Clean synthesis at the full frozen spec** with real, hardened timing analysis: WNS +4.505 ns at 100 MHz, 17% LUT / 14% FF / 26% DSP / 16% BRAM utilization — comfortable headroom on every axis, though see the caveats in Section 6.5 about what this figure does and does not prove.
- **A written, versioned interface contract** for the collaborator's corner-turn block, including the address formula, capacity analysis, ping-pong requirement, and — newly, per Section 4.4 — the clock-domain-crossing requirement, so integration risk is documented ahead of time rather than discovered during bring-up.
- **A build process hardened against its own two worst historical failure modes**: it is now structurally impossible for the build scripts to silently target the wrong FPGA part (Bug 4) or to report a blank/meaningless WNS as if it were a passing timing result (Bug 13).
- **A clean, reorganized repository** (docs in docs/, superseded work retained but isolated in attic/, regenerable tool output excluded via .gitignore) verified after cleanup by a full from-scratch rebuild and re-run of the unit-test and smoke-test suites, both still passing identically.
- **A NumPy golden model correctly parameterized to the frozen spec**. scripts/golden_model_3d_fft was found (Bug #14) to still be running at a stale 512x128x4, corrected to 1024x256x4, and re-verified to run cleanly end-to-end at the real dimensions (16.78 MB complex128 output cube).

9. What Is Pending — Stated Plainly

This section is deliberately exhaustive rather than reassuring; each item below represents a real, currently-open gap between the reported “working” state and a hardware-proven, production-ready design.

9.1 The corner turn itself — the largest remaining item

- **No real DDR-backed corner-turn RTL exists yet.** Only a behavioral placeholder (`ctm_stub.sv`) has been tested, and only its simulation body implements a real transpose; the synthesis body is a 1-deep register that performs no transpose at all.
- **Clock-domain crossing is unimplemented.** The requirement (100 MHz `aclk` <-> 200 MHz MIG `ui_clk`, Section 4.4) has been documented but no async-FIFO or equivalent CDC-safe RTL has been written, and the existing acceptance test (`tb_ctm_transpose.sv`) is single-clock and explicitly cannot exercise this — a separate CDC-specific test-bench and (for the physical design) CDC-aware constraints (e.g. `set_max_delay -datapath_only`) are both still needed.
- **MIG integration is unwired.** The MIG DDR3 controller has been generated (`ctm_test/`) but is not yet connected to any corner-turn RTL or to the rest of the pipeline.

9.2 Verification gaps

- **Full-scale functional simulation has never been run.** All passing functional results (Section 6.1–6.2) are at a reduced 16x16 test size, not the full 1024x256 spec, because the placeholder’s simulation array model cannot elaborate at full size (the same ~1,000,000-bit ceiling that motivated Bug 8’s fix). The fixed 1024-point Hanning window ROM also does not scale down with transform size, so reduced-size runs are structurally meaningful (protocol, addressing, control-flow) but are **not** numerically representative of full-scale behavior. Full-scale verification is blocked on the real corner turn landing, since that is what removes the array-size ceiling that forces the reduced-size test today.
- **BLK_EXP-across-varying-chirps is untested.** `ctm_stub.sv` latches `BLK_EXP` from only the first chirp of a frame and reuses it for the whole frame, which is documented as a requirement but rests on the assumption that every chirp in a cube produces the same BFP exponent from the Range FFT. This is a reasonable assumption for a real coherent radar scene, but it is not a hardware guarantee (BFP scaling is chosen adaptively per block), and the current constant-input smoke test cannot exercise it at all, since a constant cube trivially has the same exponent on every chirp. This needs validation once realistic, non-constant, ADC-like test data is available.
- **Real DDR bandwidth on the KC705 is unmeasured.** The 819 MB/s requirement (Section 4.3) and the conclusion that it is comfortably achievable are extrapolated from PYNQ-Z2 hardware measurements (1541 MB/s, on a *weaker* board), not measured on the KC705’s own DDR3 + MIG path.
- **Place-and-route has never been run.** All synthesis numbers (Section 6.5) are post-synthesis only; the 182 MHz `Fmax` estimate is explicitly an upper bound that will shrink once routing delay is included, and final resource utilization is not yet known because it does not include the real corner-turn logic.

9.3 System-level items not yet started at all

- **Board-level top-level module.** No top-level wrapper exists that connects to KC705 physical pins, the 200 MHz differential system clock, or a board-level reset. The frozen out-of-context XDC (Section 6.5) deliberately omits all pin assignments because they are

meaningless for out-of-context synthesis of the DSP block alone; a real board-level XDC (distinct from the historically-broken `rtl/kc705_timing.xdc`, which still has the wrong port names and a 250 MHz clock left over from an earlier, unrelated context) is a separate future task.

- **ADC input interface.** Nothing has been built to actually bring real ADC samples onto the 128-bit packed bus format the pipeline expects — the current testbenches inject synthetic vectors directly.
- **Output interface.** No mechanism exists yet to get CFAR detections off the FPGA (e.g. to a host, to DDR for later readout, or over a physical interface).
- **Bitstream generation and hardware bring-up.** The design has never been placed, routed, bitstream-generated, or programmed onto a physical KC705 board. Every result in this report is either simulation or post-synthesis static analysis.

9.4 9.4 Noted but deliberately not yet acted on

- **BRAM optimization opportunity.** All 8 generated xfft core instances (4 Range + 4 Doppler) are the *same* IP, generated once at a 2048-point maximum, so BRAM usage is fixed at generation time and does not change with the runtime NFFT selection — confirmed directly by observing that BRAM usage did not change between an earlier 128-point Doppler configuration and the current 256-point one. The four Doppler lanes are each carrying the BRAM cost of a 2048-point transform to perform a 256-point one. Generating a smaller, dedicated Doppler-only IP core would reclaim a real fraction of the 146 RAMB18 currently used — judged worth doing once the real corner turn’s own BRAM budget is known, so as not to optimize prematurely against an incomplete picture of total device usage.

10 10. Corner Cases and Known Limitations (Consolidated)

This section pulls together every corner case and known limitation surfaced anywhere in the project, in one place, since several were noted individually in the sections above.

1. **Reduced-size test runs are not numerically representative.** The Hanning window ROM is fixed at 1024 points regardless of the transform size actually configured; running a 16-point transform against a 1024-point window is a real, understood distortion (documented range-bin leakage was observed and correctly attributed to exactly this, not misdiagnosed as a bug).
2. **Angle-transform output for a constant/near-constant scene concentrates entirely in bin 0** with bins 1-3 exactly zero — correct behavior, but worth stating explicitly since it can look like “the angle FFT isn’t doing anything” to someone unfamiliar with the math of a 4-point DFT of a DC input.
3. **BLK_EXP is latched only from the first chirp of a frame** and assumed constant across the frame (Section 9.2) — correct for a coherent, roughly-constant-amplitude scene, unverified for cases where per-chirp target amplitude actually varies enough to change the Range FFT’s chosen exponent mid-frame.
4. **The 128-bit 4-lane packing assumes strict antenna lockstep.** If any future modification ever allowed the four antenna streams to be handled independently (e.g. for a partially-populated array, or a future variable-antenna-count mode), the entire justification for treating the Angle FFT as a zero-cost combinational block (Section 3.2) would need to be revisited, since it depends on all four antennas of a cell always arriving on the same bus beat.
5. **Corner-turn read/write access pattern is bandwidth-critical, not just capacity-critical.** The measured 252x bandwidth swing between 128-byte and 16 MB DDR transfers (Section 4.3) means a functionally-correct but naively-implemented (row-at-a-time) real corner turn could pass every functional test yet fail to meet the 819 MB/s requirement by two orders of magnitude. This is a case where “the transpose is logically correct” is necessary but not sufficient.
6. **The corner-turn placeholder’s two bodies (`#ifdef SYNTHESIS`) are functionally different modules that happen to share a file** — the synthesis body performs no transpose at all (Section 3.3, item under “corner-turn interface contract”). Anyone reading only the synthesis-path resource/timing numbers (Section 6.5) without this context could mistakenly believe the corner turn itself has been proven to fit and meet timing, when in fact only a 1-deep register standing in for it has been.
7. **A blank/missing WNS value must always be treated as “timing not analyzed,” never as “timing passed.”** This was directly observed as a real failure mode in this project (Bug 13) and is now defended against in tooling (Section 7), but the underlying corner case — Vivado can complete synthesis with 0 errors while having performed no timing analysis whatsoever if the XDC fails to bind — is general and worth remembering for any future constraint-file change.
8. **A “passing” testbench that only checks beat/transaction counts, not data values, will pass against a completely broken data path.** Directly demonstrated by Bug 2/3 (two independently broken transpose implementations both would have passed the original beat-count-only test). All current testbenches have since been strengthened to check actual values, but this is recorded as a standing principle for any future testbench added to the project, including the eventual CDC-specific corner-turn test.
9. **$N \bmod 8 == 0$ framing constraint from the earlier Stage-1 spec.** The original 256-bit-bus, 8-bins-per-beat framing rule used in the Stage-1-only specification cannot express a partial beat, and a hypothetical future 4-point spatial-only transform (4 bins = half a beat) would violate it. This does not affect the current 1024x256x4, 128-bit-bus design directly, but is retained here since it was explicitly flagged as a future integration hazard

in the earlier frozen specification and should not be forgotten if the bus width or framing scheme is ever revisited.

11 11. Repository Structure (Current State)

```
rtl/                synthesizable RTL (radar_dsp_3d_top.sv and all sub-blocks)
tb/                 testbenches (tb_radar_dsp_3d_top_smoke.sv, tb_ctm_transpose.sv, ...)
scripts/           build, elaborate, simulate, cleanup (Tcl + PowerShell)
constraints/        XDC files (ooc_radar_dsp_3d_top.xdc is the current, correct one for OOC runs)
vivado_ip_kc705/    generated xfft_0 IP -- every build/synth script depends on this
ctm_test/           generated MIG DDR3 IP for the real corner turn (not yet wired in)
docs/               design notes and planning documents
attic/              superseded RTL and stale results, kept intentionally for project history
```

Reproduction commands:

```
$env:Path += ";C:\Xilinx\2025.1\Vivado\bin"
```

```
.\scripts\run_unit_tests.ps1           # seconds, no IP needed
vivado -mode batch -source scripts/run_pipeline_smoke.tcl  # end-to-end, real IP, reduced si
vivado -mode batch -source scripts/check_elab.tcl          # elaboration, full 1024x256x4 s
vivado -mode batch -source build_synth.tcl                 # synthesis (requires floating l
```

12 12. Summary Assessment

What is proven, with evidence: the entire pipeline *except the corner turn* is functionally correct at reduced scale against real Xilinx IP, numerically correct where checked (exact conjugate symmetry, exact angle-transform zeros), free of the specific deadlock class that was found and fixed, elaborates and synthesizes cleanly at the full frozen 1024x256x4 spec, and closes 100 MHz timing with 45% margin at the post-synthesis stage.

What is not yet proven: anything involving the real corner turn (correctness of an actual DDR-backed transpose, the clock-domain crossing, and real DDR bandwidth under realistic access patterns), full-scale (as opposed to reduced-scale) functional correctness, post-place-and-route timing and utilization, and anything to do with physical hardware — no bitstream has been generated and nothing has been programmed onto a board.

The single largest remaining risk is the corner-turn block: it combines a memory-capacity problem (must be DDR-backed), a bandwidth/access-pattern problem (252x measured sensitivity), and — as of this reporting cycle — a previously-undocumented clock-domain-crossing problem, all in one block that is being built by a different engineer against a written contract rather than through direct collaboration on shared code. The contract has been kept current (Section 3.3 and Section 4.4) specifically to manage that risk, but the block itself remains unbuilt and unverified.

13 13. Technical Appendix — Formulas, Derivations, and Block-Level Detail

This appendix goes one level below Sections 1-12 into the actual arithmetic, bit widths, and equations implemented in RTL, taken directly from the source files (rtl/*.sv) and build scripts (scripts/*.tcl, build_synth.tcl), not re-derived from memory. Every formula below is quoted or reconstructed from a specific file so it can be checked against the source directly.

13.1 13.1 Data cube size and bandwidth — full derivation

Cube size.

```
bytes per cell      = N_ANT x (I + Q) = 4 x (2 B + 2 B) = 4 x 4 B = 16 B
cube size           = N_RANGE x N_CHIRP x bytes_per_cell
                    = 1024 x 256 x 16 B
                    = 4,194,304 B = 4.19 MB (matches rtl/ctm_stub.sv comment exactly)
ping-pong cube      = 2 x cube size = 8.39 MB
```

This is the same 4.19 MB arrived at two different ways in the source: PROJECT_LOG.md computes it as $1024 \times 256 \times 4 \text{ ant} \times 4 \text{ B} = 4.19 \text{ MB}$ (4 bytes = one complex Q1.15 sample per antenna), while ctm_stub.sv computes it as $1024 \times 256 \times 16 \text{ B}$ (16 bytes = all 4 antennas' complex samples per cell). Both reduce to the identical 4,194,304 bytes; they are just different groupings of the same multiplication, and the two independent derivations agreeing is itself a sanity check.

On-chip memory comparison.

```
XC7K325T (KC705)   Block RAM = 445 x RAMB36 = 445 x 36 Kb = 16,020 Kb = 2.00 MB
XC7Z020 (Zybo)     Block RAM = 140 x RAMB36 = 140 x 36 Kb = 5,040 Kb = 0.63 MB (rounded to 0.6 MB in th
```

8.39 MB (ping-pong cube) vs. 2.00 MB (KC705 total BRAM) -> cube is 4.2x the ENTIRE device's BRAM

This is why “tile it cleverly on-chip” was rejected outright rather than attempted: even using **100% of the device's Block RAM for nothing but the corner turn** would provide less than a quarter of the required ping-pong capacity.

Bandwidth requirement.

```
BW = 2 x bytes_per_sample x N_rx x f_sample
    = 2 x 4 B x 4 x 25.6 MSPS
    = 819.2 MB/s (rtl/ctm_stub.sv states 819 MB/s; the extra 0.2 rounds off)
```

The leading factor of 2 accounts for the corner turn's two independent traffic directions — one write (Range FFT output arriving) and one read (Doppler FFT input being drained) happening concurrently in the ping-pong scheme. bytes_per_sample = 4 B is one complex Q1.15 sample (16-bit I + 16-bit Q) for one antenna. N_rx = 4 antennas. f_sample = 25.6 MSPS is the per-antenna range-sample rate implied by the frame rate below.

Why N_range and N_chirp cancel out of the bandwidth formula. Frame rate and per-frame bytes move in exactly opposite directions as transform size changes:

```
frames/s      = f_sample / N_range (one frame = one full range sweep per chirp... )
bytes/frame    = N_range x N_chirp x bytes_per_cell
BW             = frames/s x bytes/frame / N_chirp (chirp-rate normalization)
              = f_sample x bytes_per_cell <- N_range and N_chirp both cancel
```

This is why doubling the range resolution (1024 -> 2048) or the number of chirps (256 -> 512) changes frame size and frame rate, but never changes the sustained bandwidth requirement — confirmed directly in SPEC_FROZEN.md’s own derived-rate table, where N=1024 gives 403.125 MB/s and N=2048 gives 401.5625 MB/s for the (different, single-lane) Stage-1 engine — nearly identical despite doubling the transform size, for exactly this reason.

Measured vs. theoretical DDR bandwidth.

MIG DDR3-1600, 64-bit bus:

theoretical peak = 64 bit x 1600 MT/s / 8 = 12,800 MB/s = 12.8 GB/s
 required = 819 MB/s
 theoretical margin = 12.8 GB/s / 819 MB/s ~ 15.6x

Measured on PYNQ-Z2 hardware (a WEAKER board than KC705):

16 MB transfers : 1539 MB/s (1.88x the requirement, real hardware, not theoretical)
 128 B transfers : 6 MB/s (0.007x the requirement -- catastrophically insufficient)
 swing : 1539 / 6 = 256.5x (reported in the log, rounded, as "252x")

The conclusion drawn from this: **the corner turn’s real risk is not raw bandwidth — it is access-pattern-induced bandwidth loss.** A functionally correct but row-at-a-time implementation could measure well below the 819 MB/s requirement despite the DRAM itself having over 15x theoretical headroom.

13.2 13.2 Q1.15 fixed-point format and windowing arithmetic

Q1.15 format. 16-bit signed, 1 sign bit + 15 fractional bits, representable range:

value = raw_int16 / 32768
 range = [-1.0, 0.999969482421875] (raw = -32768 .. +32767)
 resolution (LSB) = 1 / 32768 ~ 3.0518e-5

Windowing multiply-and-rescale (rtl/window_lane.sv). The Hanning coefficient ROM is Q1.15 (16-bit), and the ADC sample is Q1.15 (16-bit). A fixed-point multiply of two Q1.15 operands produces a Q2.30 product:

mult_i = data_i_q * coeff_q // Q1.15 x Q1.15 -> Q2.30, 32-bit signed result

To return to Q1.15 for the FFT core’s input, the code takes bits [30:15] of the 32-bit product — i.e. it discards the top sign-extension bit and the bottom 15 fractional bits, keeping the 16 bits that represent the Q1.15-scaled result:

m_axis_tdata = {mult_q[30:15], mult_i[30:15]} // straight truncation, not rounded

This is a **truncating**, not rounding, bit-slice — worth flagging as a corner case: on the analog-quantization scale it introduces at most 1 LSB of downward-biased error per multiply, which is a minor and standard cost of fixed-point windowing, but it is a real (if small) source of numerical difference from the NumPy floating-point golden model, and should not be mistaken for a bug if a bit-exact match against the golden model is ever attempted.

Hanning window ROM size. Fixed at hanning_rom[0:1023] (1024 entries, 16-bit each = 2 KB), addressed by a 10-bit counter (addr_cnt, logic [9:0]) that wraps at TLAST. This ROM does **not** resize with the runtime-configurable transform length — it is always 1024 entries regardless of whether the core is running a 1024-point or a reduced-size transform, which is the exact mechanism behind the “reduced-size test = not numerically representative” limitation in Sections 6.1 and 10.

13.3 13.3 Range/Doppler FFT core configuration (Xilinx xfft v9.1)

Generated once (scripts/create_fft_ip.tcl) and shared between the Range and Doppler stages via runtime reconfiguration:

CONFIG property	Value	Meaning
transform_length	2048	Maximum size the core is physically built for — required even though Range only runs at 1024, because
run_time_configurable_transform_length	transform_length	runtime-configurable mode can only shrink below this ceiling, never exceed it
implementation_options	pipelined_streaming_io	Enables the NFFT config-channel field to select 1024 or 2048 (Range) / 128 or 256 (Doppler) at runtime without regenerating the IP
data_format	fixed_point	1 sample/clock throughput, frozen spec §4
input_width / phase_factor_width	16 / 16	—
scaling_options	block_floating_point	Q1.15 data and twiddle factors, frozen spec §4/§25
rounding_modes	convergent_rounding	Frozen spec §4/§8
output_ordering	natural_order	Round-half-to-even, frozen spec §4/§9
complex_mult_type	use_mults_performance (4-mult)	Frozen spec §4
		Resolves spec gap G1 from SPEC_FROZEN.md — the project settled on the 4-multiplier structure rather than the 3-multiplier resource-saving option, prioritizing consistent rounding/latency over the ~25% DSP saving the 3-mult option would have offered

CONFIG property	Value	Meaning
number_of_stages_using_block_ram_valid_data_range	5 (of a valid data range)	For N=2048 the internal butterfly stage delay-line lengths are 1024, 512, 256, 128, 64, 32, ... down to 1; setting this to 5 routes the 5 <i>largest</i> delay lines (1024+512+256+128+64 = 1984 of 2047 total words) into Block RAM rather than distributed-RAM/SRL, because BRAM was the resource with more headroom relative to DSP at the time this was set (140 RAMB36 vs. 220 DSP48E1 on the smaller Zybo device originally evaluated)

NFFT select-code mapping (rtl/fft_config_fsm.sv, parameters NFFT_SEL0/NFFT_SEL1, config word bits [4:0]):

Range stage: nfft_sel_i=0 -> NFFT=10 ($2^{10}=1024$) nfft_sel_i=1 -> NFFT=11 ($2^{11}=2048$)
Doppler stage: nfft_sel_i=0 -> NFFT=7 ($2^7=128$) nfft_sel_i=1 -> NFFT=8 ($2^8=256$)

The frozen spec's DOPPLER_SEL=1'b1 therefore selects NFFT=8 (256-point) — this is the exact bit that Bug #12 (Section 7) found left at its default 1'b0 (128-point), and it is why radar_dsp_3d_top.sv's module defaults were changed and an elaboration-time assertion added.

Config channel gating rule. fft_config_fsm only re-issues a configuration write when the requested size changes **and** no frame is currently active (!frame_active_i) — a mid-frame NFFT write would corrupt the transform in progress. This directly answers spec gap G9 from SPEC_FROZEN.md (“runtime size changes must be fenced at a frame boundary — confirm the RTL enforces this”): confirmed, this is exactly what the FSM does.

13.4 13.4 Angle FFT — full closed-form 4-point DFT (rtl/angle_fft4_par.sv)

The Angle stage is not a generic FFT core; it is a hand-derived, fully combinational 4-point radix-2 DFT, chosen because it costs zero DSP and zero BRAM when the 4 antenna samples already arrive on the same bus beat (Section 3.2). Twiddle factor for N=4 is $W = e^{(-j*2*\pi/4)} = -j$, so the derivation reduces to pure add/subtract and a 90-degree rotation (swap-and-negate):

Stage 1 (butterfly):

```

s0 = x0 + x2      d0 = x0 - x2
s1 = x1 + x3      d1 = x1 - x3

```

Stage 2 (combine, using $-j*(a+jb) = (b,-a)$ and $+j*(a+jb) = (-b,a)$):

```

X0 = s0 + s1      (bin 0, DC / sum of all antennas)
X2 = s0 - s1      (bin 2, Nyquist of the 4-point transform)
X1 = d0 - j*d1    = (d0_re + d1_im, d0_im - d1_re)      (bin 1)

```

$X3 = d0 + j*d1 = (d0_re - d1_im, d0_im + d1_re) \quad (\text{bin } 3)$

Bit growth and rescaling. Inputs are 16-bit signed Q1.15. Each add/subtract can grow the width by 1 bit, so after two butterfly stages the raw sums are up to 18-bit signed (X_re/X_im declared signed [17:0]). A 4-point DFT has processing gain of exactly 4 (2 bits), so the design rounds and right-shifts by 2 to return to Q1.15:

```
scale16(v) :   r = (v >>> 2) + round_bit           // round-half-
up on bit [1] before the shift
              saturate r to [-32768, 32767]
              return r[15:0]
```

and the `BLK_EXP` carried in `tuser` is incremented by exactly 2 ($u_r \leq s_axis_tuser + 8'd2$) to record that the mantissa was divided by 4 — this is the +2 referenced in the exponent-accumulation chain in Section 2.4. The whole operation — unpack, butterfly, combine, rescale, register — completes in a single clock cycle, which is only possible because there is no time-multiplexing across antennas to sequence.

13.5 13.5 Magnitude-squared arithmetic (rtl/complex_mag2.sv)

$|X|^2 = Re^2 + Im^2$

Width derivation, worst case. `Re` and `Im` are each signed Q1.15, so the most negative representable value is $-32768 = -2^{15}$. Its square is:

```
(-2^15)^2 = 2^30
worst-case sum = 2^30 + 2^30 = 2^31
```

2^{31} requires 32 bits to represent as an **unsigned** value (it does not fit in 31 bits, let alone a signed 31-bit field) — this is exactly the reasoning behind freezing the output as 32-bit unsigned in `SPEC_FROZEN.md` §7, and is recorded in the source as “spec error #4.” The RTL implements this as a clean 3-stage pipeline (capture -> two independent multiplies -> add), specifically to avoid the earlier bug (Bug #6, Section 7) where the same squares were computed twice, wasting up to 2 extra DSP48 multiplier instances per lane.

Exponent doubling. `tuser` (carrying `BLK_EXP`) passes through this block completely unchanged in value, but its *meaning* changes: prior to this block the true scale factor is 2^{BLK_EXP} applied to a complex number; after squaring, the true scale factor of the output is $2^{(2*BLK_EXP)}$. The RTL does not adjust the `tuser` value itself — the consumer is required to know to double it. This directly answers spec gap G8 from `SPEC_FROZEN.md` (“`BLK_EXP` semantics after `magsq` ... undocumented”): it is now documented at the point of use, in the module’s own header comment.

13.6 13.6 CA-CFAR detector arithmetic (rtl/ca_cfar.sv)

Parameters: `LEAD_CELLS=4`, `TRAIL_CELLS=4`, `GUARD_CELLS=2` (defaults). Derived:

```
REF_CELLS      = LEAD_CELLS + TRAIL_CELLS = 8
TOTAL_WINDOW   = REF_CELLS + 2*GUARD_CELLS + 1 = 8 + 4 + 1 = 13 cells
CUT_INDEX      = LEAD_CELLS + GUARD_CELLS = 6           (Cell Under Test sits exactly in the middle)
```

The window is implemented as a 13-deep shift-register pipeline (`window_pipe[0:12]`), not a RAM, because the whole window must be visible combinationally for the noise-sum reduction — this is a deliberate area/simplicity trade-off (`window_pipe` costs $13 \times 32\text{-bit} = 416$ flip-flops per lane, times 4 lanes) that was accepted rather than built as a more compact but more complex dual-port RAM design.

Noise estimate.

```
noise_sum = sum(window_pipe[0 .. LEAD_CELLS-1])           // leading reference cells
           + sum(window_pipe[CUT_INDEX+GUARD_CELLS+1 .. TOTAL_WINDOW-
1]) // trailing reference cells
           (guard cells and the CUT itself are excluded from the sum)
```

```
noise_average = noise_sum >> REF_SHIFT,  REF_SHIFT = clog2(REF_CELLS) = clog2(8) = 3
```

Dividing by a power-of-two reference-cell count via a right-shift, rather than a real divider, is why REF_CELLS (i.e. LEAD_CELLS + TRAIL_CELLS) **must** be a power of two — enforced at simulation time by a compile-time-style \$error check ((1 << REF_SHIFT) != REF_CELLS). This shift-amount derivation is exactly the fix for Bug #7 (Section 7), which had this hardcoded to a fixed shift-by-3 regardless of the actual parameters.

Detection decision.

```
dynamic_threshold = noise_average * threshold_scale_i    // threshold_scale_i is a 16-
bit runtime Pfa-scaling factor
target_detected   = (cut_value > dynamic_threshold) AND valid
```

threshold_scale_i is left as a runtime input specifically so the false-alarm-rate trade-off can be tuned without resynthesis — no fixed Pfa formula (e.g. the standard CA-CFAR $\alpha = N \cdot (Pfa^{(-1/N)} - 1)$ scale-factor derivation) is implemented in hardware; that computation is assumed to happen in software/firmware, which then writes the resulting threshold_scale_i value into the core.

13.7 13.7 Block-floating-point exponent accumulation, exact bit path

Putting Sections 2.4, 13.4, and 13.5 together, the full exponent path from ADC input to detection output is:

```
E_range      : reported directly by the Range xfft core's BFP scaling
E_doppler    : reported directly by the Doppler xfft core's BFP scaling (overwrites tuser)
               -> exp_accum.sv holds E_range across the Doppler core's ~2N-
cycle latency
               so it is not lost when tuser is overwritten, then the two are summed:
E_range+doppler = E_range + E_doppler
E_angle        = E_range + E_doppler + 2      (angle_fft4_par's fixed +2 from its >>2 rescale)
E_final (|X|^2) = 2 * (E_range + E_doppler + 2) (complex_mag2 doubles the exponent, but does NOT
                                                encode this in tuser --
the +1 doubling is implicit,
the consumer must apply it, per Section 13.5)
```

```
true magnitude-squared value = raw_output_int32 x 2^E_final
```

exp_accum's FIFO depth is DEPTH = 2^{DEPTH_LG} frames (default DEPTH_LG=2 -> depth 4), sized to exceed the maximum number of frames that can be in flight simultaneously across the Doppler stage's pipeline latency; an ovfl_o sticky flag exists specifically so a depth that turns out to be insufficient produces an explicit, catchable fault rather than a silently wrong exponent being handed out.

13.8 13.8 4-lane packing address formula and DDR layout (rtl/axis_pack4.sv, rtl/ctm_stub.sv)

Bus packing. Lane L (antenna L) occupies bits [32*L +: 32] of the 128-bit bus, for L = 0..3. This is the “physical” layout inherited directly into the DDR burst layout the corner turn writes:

$\text{ddr_addr} = ((\text{chirp} * \text{N_RANGE}) + \text{range}) * \text{N_ANT} + \text{antenna}$

i.e. antennas are stored contiguously in DRAM for a fixed (range, chirp) cell, so a single burst read/write handles all 4 antennas of a cell at once — this is the physical realization of the “antennas must stay in lockstep” requirement from Section 2.3/3.2, expressed as an actual memory address formula rather than just a timing rule.

Corner-turn transpose address function (ctm_stub.sv, shared by both the write-side range-major access and the read-side chirp-major access):

$\text{addr}(c, r) = c * \text{N_RANGE} + r$ where $c = \text{chirp index } [0, \text{N_CHIRP})$, $r = \text{range index } [0, \text{N_RANGE})$

Writing proceeds range-major (fill all r for a fixed c , matching the order Range-FFT output naturally arrives in), and reading proceeds chirp-major (fill all c for a fixed r , the order the Doppler FFT needs) — both indexing into the *same* linear memory using the *same* formula, which is precisely what makes this a **transpose**: two different traversal orders over one address space. This is the formula Bug #2 (Section 7) originally got wrong in two independent ways (one implementation didn’t transpose at all; the other used two different, mismatched formulas for write vs. read that only coincidentally agreed at 2 of 65,536 possible addresses).

13.9 13.9 Synthesis resource-count cross-check

The reported 216 DSP48E1 slices (Section 6.5) is not an unexamined tool output — it was cross-checked by hand against the expected per-block DSP cost:

xfft cores	: 8 instances (4 Range + 4 Doppler), each xfft v9.1 configured for use_mults_performance (4-mult complex multiplier) at N=2048 max	
	-> approximately 24 DSP48E1 per core x 8 = ~192	
Fabric logic	: window_lane x4 (1 real multiply each, I and Q share... 2 DSP each) = 8	
	complex_mag2 x4 (2 multiplies each: Re^2, Im^2) = 8	
	ca_cfar x4 (1 multiply each: $\text{noise_average} * \text{threshold_scale}$) = 8	
	fabric subtotal = 24	

TOTAL		192 + 24 = 216 <-

matches exactly

The exact match between the bottom-up hand estimate and the tool-reported 216 is treated in this project as corroborating evidence that the utilization report describes the design actually intended, rather than accepted at face value — consistent with the project’s general pattern (Section 7.1) of not trusting a single tool output without an independent cross-check.

13.10 13.10 Timing budget arithmetic

target clock period	= 10.000 ns (100 MHz, set in constraints/ooc_radar_dsp_3d_top.xdc)
reported WNS	= +4.505 ns
implied critical path	= 10.000 - 4.505 = 5.495 ns
implied Fmax	= 1 / 5.495 ns = 181.98 MHz ~ 182 MHz (post-synthesis, pre-route)
required Fmax (headroom)	= 100 MHz clock target vs. 182 MHz achieved -
> 82% margin in frequency terms,	
or equivalently 45% margin in period/slack terms (4.505 ns of a 10.000 ns budget)	

Both the “45% margin” and “182 MHz” figures quoted in Section 6.5 are therefore two different unit expressions (time-domain slack fraction vs. frequency-domain headroom) of the same single measured critical-path delay, not two independent measurements.

13.11 13.11 Clock-domain-crossing frequency relationship (Section 4.4, restated arithmetically)

DDR3-1600 memory clock = $1 / 1250 \text{ ps} = 800 \text{ MHz}$ (MIG TimePeriod=1250)
PHYRatio = 4:1
ui_clk = $800 \text{ MHz} / 4 = 200 \text{ MHz}$
DSP pipeline clock = 100 MHz (aclk)
ratio = $200 \text{ MHz} / 100 \text{ MHz} = 2.000$ (exactly 2x, not an odd/asynchronous ratio)

Bandwidth check across the crossing:

ui_clk side : 64-bit bus x 200 MHz = 12,800 MB/s theoretical (matches Section 13.1)

aclk side : 819 MB/s required

-> the crossing must MOVE data reliably across two clock domains; it has no bandwidth problem, only a synchronization-correctness problem (metastability, data-coherency across the boundary)

The exact 2:1 ratio is worth noting explicitly: because it is a clean integer ratio (not, say, 187 MHz vs. 100 MHz), a synchronous or mesochronous CDC technique might eventually be viable as an alternative to fully asynchronous FIFOs — but this has not been evaluated, designed, or verified in any form, and remains purely an observation for whoever implements the real corner-turn block to consider, not a recommendation acted upon in this project.